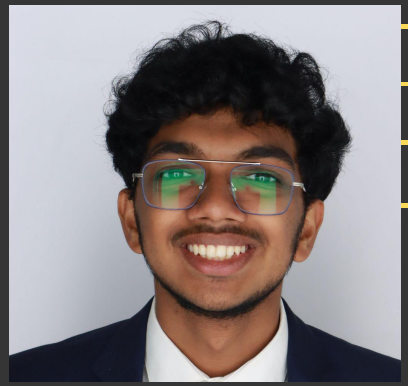
ARM

NNAS for RF Modulation Classification

Ashwin Menon

Team Members



Ashwin Ranjith Menon
Team Member

Contact:
menonar@mail.uc.edu

Dr. Boyang Wang
Faculty Advisor

Contact:
wang2ba@ucmail.uc.edu

Goals

With this project, we're exploring reinforcement learning to perform neural network architecture search which will improve radio frequency modulation classification on constrained hardware. Deep learning models for RF modulation deliver strong accuracy but require high performance devices, making them impractical for embedded and edge devices. We will be using the Q-learning approach to generate neural networks with small parameter counts that preserve accuracy

Intellectual Merits

The novelty of this work lies in applying reinforcement learning (based on the Q-learning approach) to generate machine learning models that reduce parameter count while maintaining classification accuracy. Instead of manually designing lightweight neural networks, by automating the generation process we can use this method to learn optimal architectures for our domain, which is efficient model design to be deployed on lightweight hardware.

Broader Impacts

This project makes machine learning more accessible by helping us deploy lightweight neural networks on even restrained hardware like Jetson Nanos. The method used in this project can also be generalized for any other usage domain - not just RF applications

Design Diagram - Level 0

Inputs

- RadioML 2021 dataset
- Constraint table defining valid layer connections
- Configuration set listing possible hyperparameters for each layer type
- Q-value table (initialized empty) to record state transition efficacy
- Replay database (initialized empty) to store created models, accuracies, and parameter counts
- Epsilon scheduler defining models per epsilon value

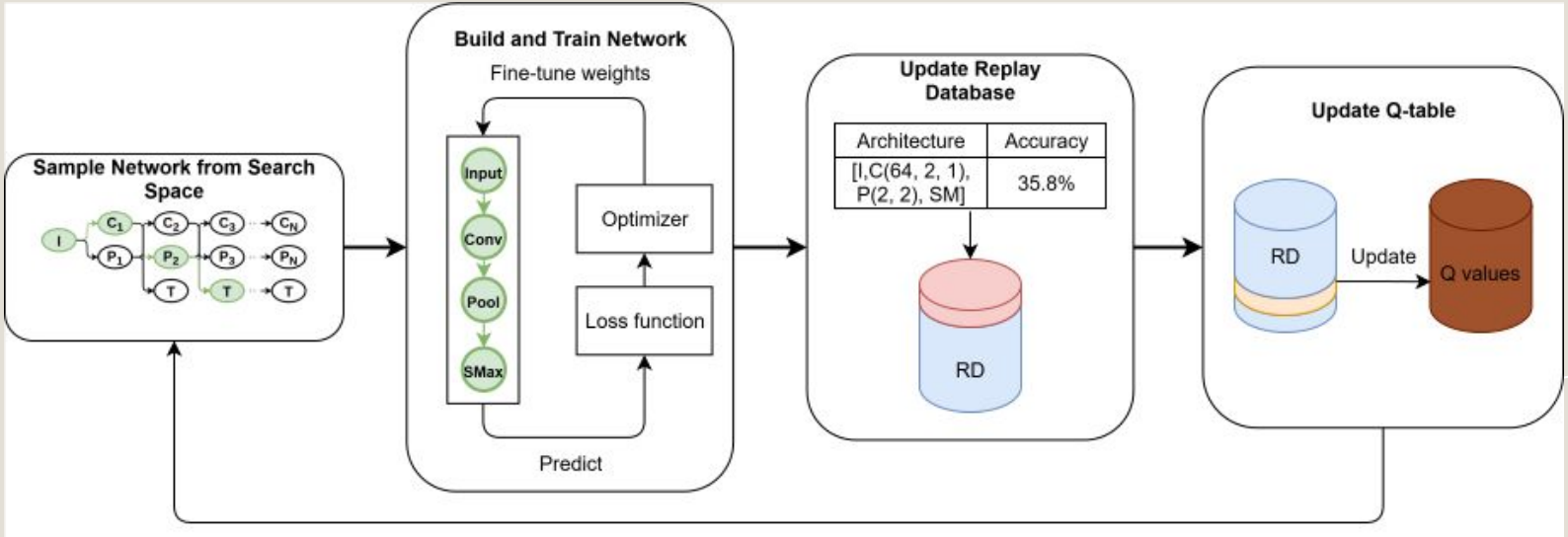
Q-learning Agent

- Build neural network using Markov Decision Process
- Train network after generation
- Record results (accuracy + trainable parameters) in Replay Database
- Sample replay database: pick random model, compute reward, update Q-values
- Repeat sampling/updating 100 times
- Follow epsilon scheduler for models per phase

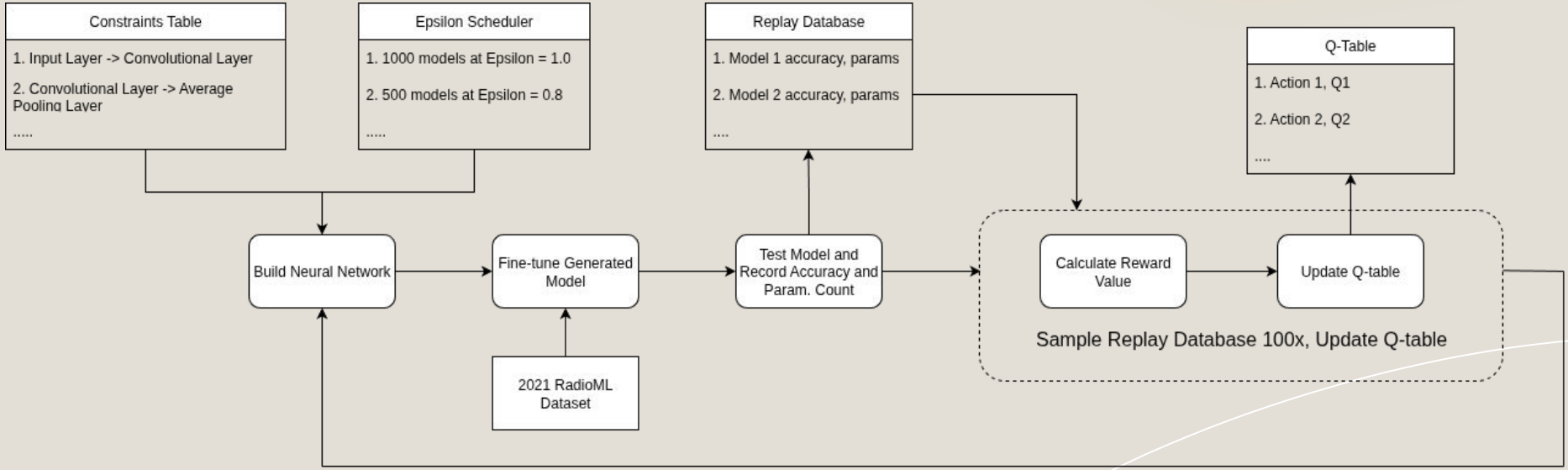
Outputs

- Q-table approximating action efficacy
- Replay database storing created models, accuracies, and trainable parameters for selecting top candidates

Design Diagram - Level 1



Design Diagram - Level 2



Technology Used

1

Hardware

- Provided by the UCDaSec Research Lab
- Processor: Titan i9
- Graphics Card (usage recommended): RTX Titan TU102

2

Software

- Tensorflow v2.1

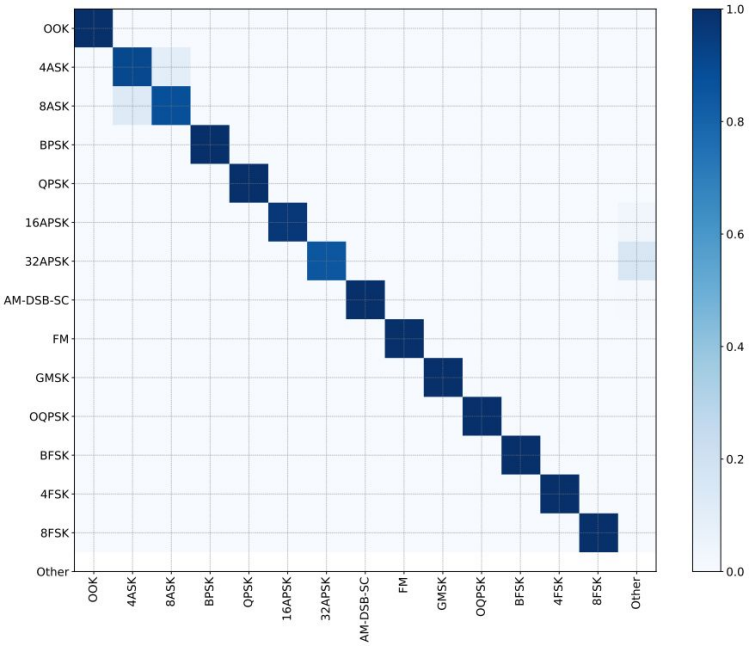
Project Milestones & Timeline

Milestone	Start Date	Completion Date
Analyze MetaQNN Research	10/13/25	10/20/25
Research RL Implementations	10/20/25	10/27/25
Procure and Analyze Dataset	10/27/25	11/10/25
Create Baseline Model	11/24/25	12/1/25
Smoke Test RL System	12/1/25	12/22/25
Conduct Small-Scale Experiments	12/22/25	12/28/25
Develop Scoring Function	12/28/25	12/30/25
Analyze Candidate Models	12/30/25	2/9/26
Code Cleanup and Pipeline Setup	2/9/26	-

Results

Model	Model Score	No. of Parameters	Parameter Reduction Rate	Size (KB)	Acc. (≥6 dB SNRs)	Acc. at 30 dB SNR	Subset Acc. (≥6 dB SNRs)	Subset Acc. at 30 dB SNR
Baseline	N/A	717,252	N/A	5,950	97.76%	98.69%	99.94%	99.96%
Pruned Baseline (h = 2)	N/A	13,981	98.20%	280	84.38%	85.33%	99.26%	99.75%
Model 312	**4.718×10^{-3}**	**11,859**	**98.37%**	**255**	**72.49%**	**73.12%**	**96.90%**	**97.53%**
Model 1802	4.695×10^{-3}	6,371	99.13%	135	68.09%	68.50%	96.59%	97.39%
Model 2171	4.638×10^{-3}	5,379	99.26%	139	70.74%	71.20%	95.89%	97.69%
Model 1452	4.423×10^{-3}	9,179	98.74%	192	68.99%	69.18%	96.27%	96.74%
Model 2482	4.406×10^{-3}	5,995	99.26%	184	69.63%	69.88%	95.48%	95.96%
Model 1004	**4.326×10^{-3}**	**2,335**	**99.69%**	**104**	**68.29%**	**68.05%**	**95.89%**	**96.10%**
Model 2609	4.298×10^{-3}	12,919	98.23%	249	69.07%	68.49%	96.01%	95.97%
Model 859	4.276×10^{-3}	10,555	98.56%	208	69.55%	69.52%	96.60%	97.04%
Model 1684	**4.168×10^{-3}**	**1,643**	**99.78%**	**103**	**66.09%**	**66.33%**	**95.49%**	**96.17%**
Model 2437	4.137×10^{-3}	5,171	99.26%	159	67.25%	66.83%	96.14%	96.66%

Results - Contd.



Division of Work

- Analyze background research paper for MetaQNN (Ashwin)
- Analyze dataset to understand and unpack it (Ashwin)
- Smoke test RL implementation to see if replay database and Q-table are being initialized properly along with saved models (Ashwin)
- Create scoring function to rank candidate models so that we can pick top-k models from the replay database (Ashwin)
- Perform cleaning up and reorganization of code if necessary. Additionally, create pipelines for reproducibility (Ashwin)

Challenges

- Had to make design decisions to constrain state space and finetune hyper-parameters to be used for neural network generation
- Also had to empirically determine the reward function for optimal convergence